{EPITECH}

SUMMER SCHOOL_

< PART 3 BOOTSTRAP - REACT STATES & EVENTS />

SUMMER SCHOOL

< WEB DEVELOPMENT: MEMORY AND INTERACTION />

> ”I think everybody in this country should learn how to program a computer because it teaches you how to think.”
> — **Steve Jobs**

Introduction

In Part 2, you learned how to build components and pass data using `props`. However, your components are currently static. They look good, but they cannot react to user clicks or remember any information.

Welcome to React **State** and **Events**.
In this bootstrap, you will learn how to make a button actually do something, and how to track what a user types in real-time.

Instead of starting from a blank page, we are going to hack the default Vite template. It already contains a working interactive button. We will analyze how it works, and then add our own features to it.

Step 1: The Default Sandbox

Open your terminal and create a new temporary project for today's experiments:

```
npm create vite@latest state-sandbox -- --template react
cd state-sandbox
npm install
npm run dev
```

Open the URL in your browser. You will see the default Vite template with the React logo and a button that says `count is 0`. Click the button. The number goes up!

Step 2: Analyzing the Magic (Numeric State)

Open `src/App.jsx` in your code editor. Let's look at how the Vite team created that button.

Look at the top of the component:

```
import { useState } from 'react'

function App() {
  const [count, setCount] = useState(0)
  // ...
```

React needs to know *when* to update the screen. If you change a normal variable (`count = 1`), React doesn't notice. To solve this, we use the `useState` hook.
- `count` is the memory variable (starts at `0`).
- `setCount` is the ONLY function allowed to change `count`.

Now look at the button in the JSX:

```
<button onClick={() => setCount((count) => count + 1)}>
  count is {count}
</button>
```

In React, we use `onClick` to listen for clicks. When clicked, it triggers the `setCount` function, increasing the memory by 1, which instantly redraws the screen!

The Golden Rule: Never modify a state variable directly. Always use its setter function. This is the *only* way to tell React to refresh the UI.

Step 3: Tracking Text (String State)

Now let's add our own feature to the template: tracking text input in real-time. We will need a new piece of memory (a String State).

1. In `src/App.jsx`, add a new state variable right under the `count` state.
2. Add an `<input>` and a `<p>` tag inside the `<div className="card">`, right below the existing button.

Modify your code to look like this:

```
function App() {
  const [count, setCount] = useState(0)
  // 1. Create a String state (starts empty)
  const [text, setText] = useState('')

  return (
    <>
      {/* ... (Logos are up here) ... */}
      <h1>Vite + React</h1>
      <div className="card">

        {/* The Default Vite Button */}
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>

        {/* 2. Our New Text Tracker */}
        <div style={{ marginTop: '20px' }}>
          <input
            type="text"
            placeholder="Type something..."
            value={text}
            onChange={(event) => setText(event.target.value)}
          />
          <p>You are typing: <i>{text || "Nothing yet..."}</i></p>
        </div>

      </div>
      {/* ... */}
    </>
  )
}
```

The Magic Formula: `onChange={(event) => setText(event.target.value)}`
Whenever you press a key, an `event` is generated. `event.target.value` grabs the exact text currently in the box and saves it into your `text` state!

Step 4: The Toggle Switch (Boolean State)

For our final feature, we want a toggle (On/Off). The best way to store On/Off data is using a Boolean (`true` or `false`).

We will use this boolean to do **Conditional Rendering** (changing what is displayed based on the state).

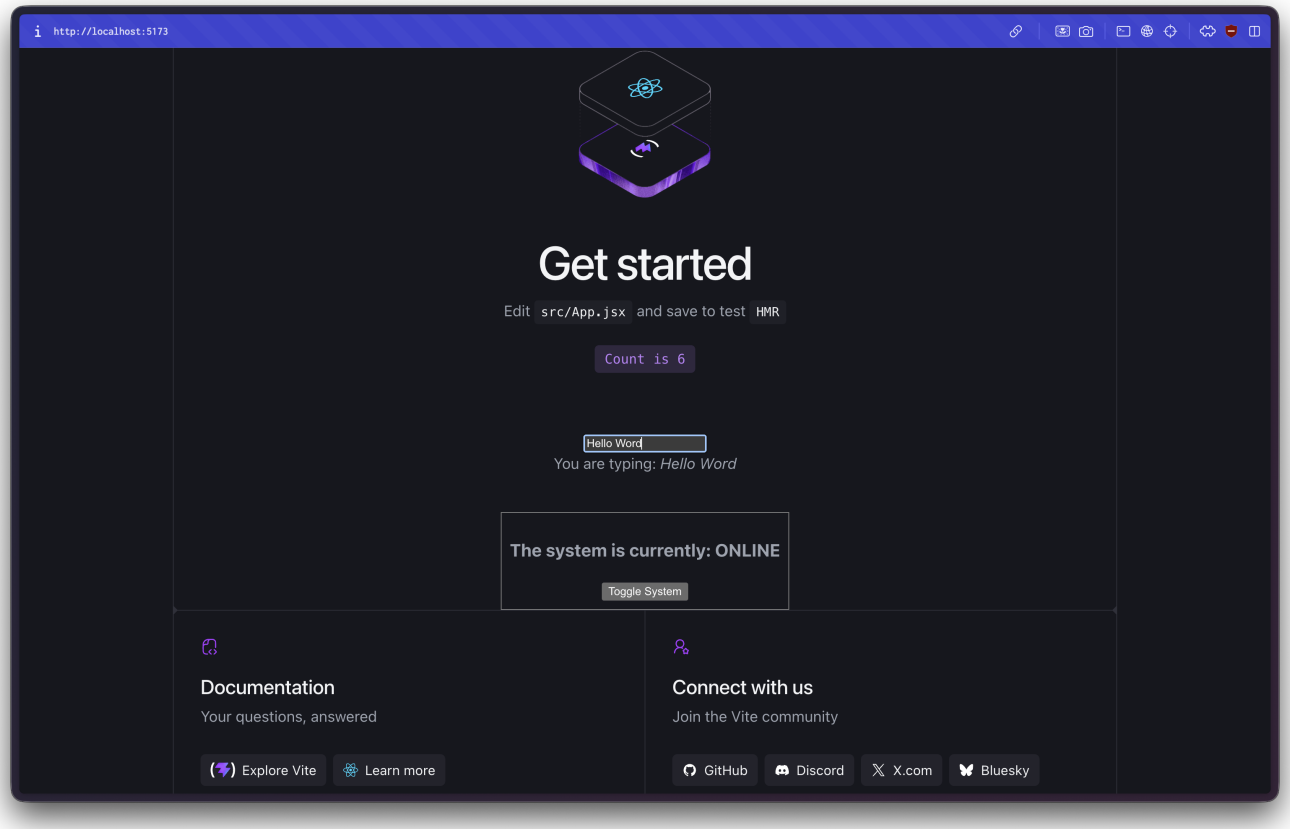
Modify your `App.jsx` one last time:

```
function App() {
  const [count, setCount] = useState(0)
  const [text, setText] = useState('')
  // 1. Create a Boolean state (starts false)
  const [isOn, setIsOn] = useState(false)

  return (
    <>
      <h1>Vite + React</h1>
      <div className="card">

        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>

        <div style={{ marginTop: '20px' }}>
          <input
            type="text"
            value={text}
            onChange={(e) => setText(e.target.value)}
          />
          <p>You are typing: <i>{text}</i></p>
        </div>

        {/* 2. Our New Toggle Switch */}
        <div style={{ marginTop: '20px', padding: '10px', border: '1px solid gray' }}>
          <h3>The system is currently: {isOn ? "ONLINE" : "OFFLINE"}</h3>

          {/* The ! operator means "Not". It flips true to false, and false to true */}
          <button onClick={() => setIsOn(!isOn)}>
            Toggle System
          </button>
        </div>

      </div>
    </>
  )
}
```

Conclusion

You now possess a complete toolkit to make your digital newspaper interactive!
- **The Counter (Numeric State)** translates perfectly to an **Article Like Button**.
- **The Text Tracker (String State)** translates perfectly to a **Footer Search Bar**.
- **The Toggle (Boolean State)** translates perfectly to the **Dark Mode** or **Unlike** bonus features!

Close your sandbox, open your main `newspaper-react` project, and adapt these logic patterns to satisfy the Editor-in-Chief!



v1